Member 1: Payment Core (Backend)

Work assignment for the first student in the 3-member team

UPI Payment Failure Diagnosis, Reconciliation and Fraud Risk Monitoring System

Your job in one line: build the ground truth of the system. The database, the payment APIs, the payment state machine, duplicate protection (idempotency) and the fake banks. Members 2 and 3 build on top of what you produce, so a correct, stable contract matters more than speed or extra features.

1. Your Role and How It Fits

The project is a simulated UPI app with two features on top: fraud-risk detection and real-time failure diagnosis. The work is split into three tracks:

Member	Track	Builds
1 (you)	backend/	Database, payment APIs, state machine, idempotency, sender and receiver bank simulators
2	fraud-intel/	Six threaded fraud checks, fraud ML model, diagnosis decision table, recovery-time estimate
3	frontend/	UPI app screens, operations dashboard, alerts, end-to-end testing

Member 2 calls into your payment flow for fraud scoring, and Member 3 calls your API from both screens. Everything that changes a payment's state goes through code you own. You do **not** write fraud scoring or UI code.

2. What You Deliver

- A running FastAPI backend with all required tables in SQLite.
- Two bank simulators (sender and receiver) whose behaviour can be switched live: success, timeout, error, lost confirmation.
- A payment state machine that rejects illegal moves and logs every change to an append-only event table.
- Duplicate-payment protection using a database unique constraint, proven under concurrent requests.
- A reconciliation path: if money is debited but the credit is missing, the payment is flagged RECONCILIATION_REQUIRED automatically.
- The endpoints other members need, matching the agreed contract exactly.
- Automated tests for all 10 payment scenarios and the concurrency test.

3. Setup

Python 3.10 or newer. Create a virtual environment inside **backend/** and install the packages:

```
pip install fastapi uvicorn sqlalchemy pydantic pytest httpx
```

Suggested layout inside backend/:

```
backend/
  app/
    db.py            engine, session, Base
    models.py        the tables
    simulators.py    sender and receiver bank behaviour
    service.py       state machine, idempotent create, payment flow
    main.py          FastAPI endpoints
    fraud_hook.py    stub that Member 2 later replaces
  seed.py            demo customers and accounts
  tests/             pytest files
```

4. Suggested Timeline

When	Phase	Your tasks	Done when
Days 1-2	0 Design lock	Draft schema, API contract, state diagram; send to Members 2 and 3 for review	Both reviewers replied and agree
Week 1	1 Skeleton	Steps 1-2: database, models, simulators, seed data	Server starts; simulators obey their mode
Week 2	2 Happy path	Steps 3-6: state machine, idempotent create, payment flow, endpoints	A POST runs a full payment and returns a state
Week 3	3 Core	Step 7: timeout watcher, concurrency proof, fraud hook	Same key x 20 threads creates one row; debit-without-credit flagged
Week 4	4 Hardening	Steps 8-9: extra endpoints, contract check, tests, logging	All 10 scenarios pass
Week 5	5 Polish	Fix bugs found by others, demo data, help with report	Guide's dry-run passes

5. Step-by-Step Tasks

Step 0. Design lock (before coding)

[ ] Write the table list and key fields in **docs/schema.md**.

[ ] Write the exact request and response JSON for each endpoint in **docs/architecture.md** (section 6 below is your starting point).

[ ] Write the state diagram (section 5, Step 3).

[ ] Send all three to Members 2 and 3 and wait for both to confirm. Changing a field name later breaks their work.

Step 1. Database and models

Use SQLite with **check_same_thread=False**, because threads will touch the database later. Create these tables:

Table	Important fields and rules
customers	id, name, email (unique), usual amount, home location
accounts	id, customer id, balance, account number (unique)
merchants	id, name, risk level (0-100)
devices	id, customer id, fingerprint, ip address, first seen
transactions	id, **idempotency_key UNIQUE NOT NULL**, sender, receiver, amount, state, location, merchant, device, timestamps
transaction_events	id, transaction id, from state, to state, detail, time. **Append-only: never update or delete rows.**
risk_results	transaction id, check name, score, reason, thread name, start, end (Member 2 fills these)
alerts	transaction id, recipient (SENDER, RECEIVER, OPS), message, time

Table	Important fields and rules
fraud_cases	transaction id, decision, total score, status (OPEN, REVIEWED, CLOSED)

Minimal engine and key models (the other tables follow the same pattern):

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

engine = create_engine("sqlite:///./check.db", connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False)
Base = declarative_base()

class Transaction(Base):
    __tablename__ = "transactions"
    id = Column(String, primary_key=True, default=new_id)
    idempotency_key = Column(String, unique=True, nullable=False)
    sender_id = Column(String, ForeignKey("accounts.id"), nullable=False)
    receiver_id = Column(String, ForeignKey("accounts.id"), nullable=False)
    amount = Column(Float, nullable=False)
    state = Column(String, nullable=False, default="INITIATED")
    created_at = Column(DateTime, default=datetime.utcnow)

class TransactionEvent(Base):
    __tablename__ = "transaction_events"
    id = Column(String, primary_key=True, default=new_id)
    transaction_id = Column(String, ForeignKey("transactions.id"), nullable=False)
    from_state = Column(String)
    to_state = Column(String)
    detail = Column(String)
    created_at = Column(DateTime, default=datetime.utcnow)
```

Step 2. Bank simulators

The simulators fake the two banks. Each call returns a status and a code. A shared config chooses the behaviour, and a small endpoint (**POST /sim/config**) changes it live so you can demonstrate every failure without restarting.

Mode	What happens	Final state you should see
SUCCESS	Responds OK after a short delay	COMPLETED
ERROR	Returns an error code	Debit error: FAILED. Credit error: RECONCILIATION_REQUIRED
TIMEOUT	Waits, then returns no response; money not applied	RECONCILIATION_REQUIRED (status check finds no credit)
LOST_ACK (receiver)	Credit really applied, but the confirmation never returns	COMPLETED (status check finds the credit)

```
import time

CONFIG = {"sender": "SUCCESS", "receiver": "SUCCESS", "delay": 0.1, "timeout": 0.5}
_credited = set()          # transaction ids the receiver bank really credited

def debit(txn_id, account_id, amount):
    mode = CONFIG["sender"]
    if mode == "ERROR":
        return {"status": "ERROR", "code": "DEBIT_FAILED"}
    if mode == "TIMEOUT":
        time.sleep(CONFIG["timeout"])
        return {"status": "TIMEOUT", "code": "NO_RESPONSE"}
    time.sleep(CONFIG["delay"])
    return {"status": "OK", "code": "00"}

def credit(txn_id, account_id, amount):
```

```
    mode = CONFIG["receiver"]
    if mode == "ERROR":
        return {"status": "ERROR", "code": "CREDIT_FAILED"}
    if mode == "TIMEOUT":                       # never applied, caller gives up
        time.sleep(CONFIG["timeout"])
        return {"status": "TIMEOUT", "code": "NO_RESPONSE"}
    if mode == "LOST_ACK":                      # applied, but confirmation lost
        _credited.add(txn_id)
        time.sleep(CONFIG["timeout"])
        return {"status": "TIMEOUT", "code": "NO_RESPONSE"}
    time.sleep(CONFIG["delay"])                 # SUCCESS
    _credited.add(txn_id)
    return {"status": "OK", "code": "00"}

def status_check(txn_id):
    return {"credited": txn_id in _credited}
```

Note: **status_check** is what lets the system tell "credit was applied but the reply was lost" apart from "credit never happened". It is the heart of the reconciliation logic.

Step 3. State machine

All state changes must go through one function that checks an allowed-transitions table and writes one event row. Nobody sets **txn.state** directly anywhere else.

From	Allowed next states
INITIATED	PROCESSING, FAILED
PROCESSING	DEBITED, FAILED, TIMEOUT
DEBITED	CREDIT_PENDING
CREDIT_PENDING	CREDITED, TIMEOUT, RECONCILIATION_REQUIRED
CREDITED	COMPLETED
TIMEOUT	CREDITED (status check finds it), RECONCILIATION_REQUIRED, FAILED
RECONCILIATION_REQUIRED	COMPLETED (resolved), FAILED (reversed to sender)
COMPLETED, FAILED	none (final)

```
ALLOWED = {
    "INITIATED": {"PROCESSING", "FAILED"},
    "PROCESSING": {"DEBITED", "FAILED", "TIMEOUT"},
    "DEBITED": {"CREDIT_PENDING"},
    "CREDIT_PENDING": {"CREDITED", "TIMEOUT", "RECONCILIATION_REQUIRED"},
    "CREDITED": {"COMPLETED"},
    "TIMEOUT": {"CREDITED", "RECONCILIATION_REQUIRED", "FAILED"},
    "RECONCILIATION_REQUIRED": {"COMPLETED", "FAILED"},
    "COMPLETED": set(),
    "FAILED": set(),
}

def transition(db, txn, new_state, detail=""):
    if new_state not in ALLOWED[txn.state]:
        raise ValueError(f"Illegal transition {txn.state} -> {new_state}")
    db.add(TransactionEvent(transaction_id=txn.id, from_state=txn.state,
                            to_state=new_state, detail=detail))
    txn.state = new_state
    db.commit()
```

Step 4. Idempotent create (duplicate protection)

Insert the row and let the **database unique constraint** decide. If the insert raises an IntegrityError, load the existing transaction and return it with a duplicate flag. Never write "check if exists, then insert", because two simultaneous requests can both pass the check. If the same key arrives with different details, reject it.

```
def create_transaction(db, key, sender_id, receiver_id, amount):
    txn = Transaction(idempotency_key=key, sender_id=sender_id,
                      receiver_id=receiver_id, amount=amount)
    db.add(txn)
    try:
        db.commit()
        return txn, True                      # newly created
    except IntegrityError:
        db.rollback()
        existing = db.query(Transaction).filter_by(idempotency_key=key).one()
        same = (existing.sender_id, existing.receiver_id, existing.amount) == \
               (sender_id, receiver_id, amount)
        if not same:
            raise ValueError("Idempotency key reused with different details")
        return existing, False                # duplicate request
```

Step 5. Payment flow

This function runs a payment from start to a final state. It checks the balance, debits, credits, and when the credit reply is missing it asks the receiver bank to confirm before deciding.

```
def process_payment(db, txn):
    sender = db.get(Account, txn.sender_id)
    if sender.balance < txn.amount:
        transition(db, txn, "FAILED", "Insufficient balance")
        return txn
    transition(db, txn, "PROCESSING")
    d = simulators.debit(txn.id, txn.sender_id, txn.amount)
    if d["status"] != "OK":
        transition(db, txn, "FAILED", f"Debit {d['code']}")
        return txn
    sender.balance -= txn.amount
    transition(db, txn, "DEBITED")
    transition(db, txn, "CREDIT_PENDING")
    c = simulators.credit(txn.id, txn.receiver_id, txn.amount)
    if c["status"] == "OK":
        db.get(Account, txn.receiver_id).balance += txn.amount
        transition(db, txn, "CREDITED"); transition(db, txn, "COMPLETED")
    elif c["status"] == "ERROR":
        transition(db, txn, "RECONCILIATION_REQUIRED", f"Credit {c['code']}")
    else:                                     # timeout: ask the receiver bank
        transition(db, txn, "TIMEOUT", "No credit response")
        if simulators.status_check(txn.id)["credited"]:
            receiver = db.get(Account, txn.receiver_id)
            receiver.balance += txn.amount
            transition(db, txn, "CREDITED", "Found by status check")
            transition(db, txn, "COMPLETED")
        else:
            transition(db, txn, "RECONCILIATION_REQUIRED", "No credit after status check")
    return txn
```

Step 6. Endpoints

Wrap the functions above in FastAPI. Validate input (positive amount, sender different from receiver, accounts exist) and return clear errors.

Endpoint	Purpose	Needed by
POST /transactions	Create and process a payment	Member 3 (UPI app)
GET /transactions/{id}	Transaction details	Member 3

Endpoint	Purpose	Needed by
GET /payment-status/{id}	Current state plus event timeline	Member 3
GET /risk/{id}	Fraud result from Member 2's data	Member 3 (dashboard)
GET /alerts, GET /cases, GET /logs/{id}	Operations dashboard data	Member 3
POST /sim/config	Switch bank simulator mode live	Demo, all members

Step 7. Timeout watcher and fraud hook (Phase 3)

- **Timeout:** run the credit call in a worker thread and wait for it with a time limit (for example 10 seconds in the demo). If it does not answer, move to TIMEOUT, run one status check, then decide COMPLETED or RECONCILIATION_REQUIRED as in Step 5. Debit-side and credit-side checks run as separate tasks so each side is verified independently.
- **Alerts:** whenever a payment becomes RECONCILIATION_REQUIRED, write rows into **alerts** for the sender (do not retry), the receiver (payment pending) and operations.
- **Fraud hook:** call Member 2's scoring function before debiting. Start with a stub (below). A decision of BLOCK moves the payment to FAILED without debiting. REVIEW lets it continue and creates a case. Confirm this rule with the team before relying on it.

```
# fraud_hook.py  (stub until Member 2 delivers the real one)
def score_transaction(txn):
    return {"decision": "APPROVE", "score": 0.0}
```

Step 8. Logging and extra endpoints (Phase 4)

- Add the alerts, cases and logs endpoints for Member 3's dashboard.
- Log every line with the transaction id so one payment can be traced from start to end.
- Optional: a reversal function that returns money to the sender for payments that end FAILED after a debit.

Step 9. Tests

Write pytest tests for every scenario in the next section, plus the concurrency test: fire the same idempotency key from 20 threads and assert exactly one row exists. A reference run of the code above gave these results:

```
success         COMPLETED              (new payment)
credit error    RECONCILIATION_REQUIRED
receiver timeout RECONCILIATION_REQUIRED
lost ack        COMPLETED              (found by status check)
debit error     FAILED
insufficient    FAILED
duplicate key   returns existing, no new debit
illegal move    COMPLETED -> PROCESSING rejected
20 threads, same key -> 1 row, 1 creator, 0 errors
```

Money check: in that run the sender started with 5000 and four payments of 1000 were debited. Two completed (receiver +2000). The other two ended RECONCILIATION_REQUIRED, so 2000 is held in limbo awaiting reversal: sender 1000 + receiver 2000 = 3000, and 3000 + 2000 held = the original 5000. If your totals do not balance like this, there is a bug.

6. API Contract (starting proposal)

Agree this with Members 2 and 3 in Step 0. After that it only changes with all three members' approval.

POST /transactions request:

```
{
  "idempotency_key": "txn-001",
```

```
  "sender_account_id": "...",
  "receiver_account_id": "...",
  "amount": 1000,
  "location": "Hubballi",       // optional
  "merchant_id": "...",         // optional
  "device_id": "..."            // optional
}
```

Response:

```
{
  "transaction_id": "...",
  "state": "COMPLETED",
  "decision": "APPROVE",
  "duplicate": false,
  "message": "Payment completed"
}
```

GET /payment-status/{id} response:

```
{
  "transaction_id": "...",
  "state": "RECONCILIATION_REQUIRED",
  "timeline": [
    {"from": "INITIATED", "to": "PROCESSING", "detail": "", "at": "2026-10-08T10:00:00"},
    {"from": "PROCESSING", "to": "DEBITED", "detail": "", "at": "..."}
  ]
}
```

Error codes: **422** invalid input, **404** unknown id, **409** idempotency key reused with different details.

Note: the reference code uses sender_id and receiver_id internally; map them to sender_account_id and receiver_account_id in the endpoint so the contract stays stable.

7. What You Give and Receive

With	You give	You receive
Member 2	Transaction object per payment; the place where fraud scoring is called; the risk_results table	A function score_transaction(txn) returning decision and score; reconciliation reason and severity for stuck cases
Member 3	Endpoints from section 6; POST /sim/config; seed data; clear error messages	Bug reports as GitHub issues when something breaks the contract
Guide	Pull requests for review; phase demos	Sign-off at each checkpoint

Unblock the others first: publish the contract, the simulators and a working POST /transactions early, even if idempotency and the full state machine are not finished yet.

8. The 10 Scenarios You Must Pass

#	Scenario	How to trigger	Expected result
1	Normal payment	Both banks SUCCESS	COMPLETED
2	Debit ok, credit fails	Receiver ERROR	RECONCILIATION_REQUIRED, alerts written
3	Debit ok, receiver times out	Receiver TIMEOUT	TIMEOUT, status check, RECONCILIATION_REQUIRED
4	Credit applied, confirmation lost	Receiver LOST_ACK	TIMEOUT, status check finds credit, COMPLETED
5	Debit fails	Sender ERROR	FAILED, balance unchanged
6	Invalid receiver	Unknown receiver account	Rejected before any debit
7	Insufficient balance	Amount above balance	FAILED, balance unchanged
8	Duplicate request	Same key twice	Existing transaction returned, one debit only
9	Concurrent duplicates	Same key from 20 threads	Exactly one transaction row
10	Blocked by fraud	Stub returns BLOCK	FAILED, no debit

9. Rules to Follow

Do

- Change state only through **transition()**.
- Let the database unique constraint enforce duplicates.
- Keep transaction_events append-only.
- Keep one database session per thread or request.
- Open a pull request for every change and ask Member 2 or 3 to review it. Do not merge your own.

Do not

- Put fraud-scoring logic in the backend. Call Member 2's function instead.
- Rename an API field without telling the other two.
- Edit another member's folder. Open an issue instead.
- Claim the system is real UPI. It is a simulation.

10. Definition of Done

Phase 0

[ ] Schema, contract and state diagram written and approved by Members 2 and 3.

Phase 1

[ ] Server starts and creates all tables.

[ ] Seed script creates sample customers and accounts.

[ ] Each simulator mode behaves as described.

[ ] POST /transactions stores a row and returns an id.

Phase 2

- [ ] A payment runs from INITIATED to a final state through the simulators.
- [ ] GET endpoints return real data including the event timeline.
- [ ] Input validation returns clear errors.

Phase 3

- [ ] Illegal transitions raise an error.
- [ ] Same key from 20 threads creates exactly one row.
- [ ] Debit without credit becomes RECONCILIATION_REQUIRED and writes alerts.
- [ ] Fraud hook is wired in; BLOCK stops the payment.

Phase 4

- [ ] All 10 scenarios pass as automated tests.
- [ ] Every endpoint matches the contract.
- [ ] Logs trace one payment end to end.

11. Questions You Should Be Able to Answer

Question	Answer
Why can money be debited but not credited?	Two banks, two databases, an unreliable network. Debit and credit cannot be one atomic step.
How do you stop duplicate payments?	A unique constraint on the idempotency key in the database. A repeat returns the existing transaction.
Why not just check if the key exists first?	Two simultaneous requests can both pass the check. The database constraint is the real guard.
What is the state machine for?	It makes illegal moves impossible and records every change, so each payment has a complete history.
What happens if the credit reply is lost?	The payment goes to TIMEOUT. A status check asks the receiver bank. If it was credited, the payment completes; otherwise it is flagged for reconciliation.
Why is the event table append-only?	It acts as an audit ledger. Rows are never changed, so the history can be trusted.
Is this real UPI?	No. Sender bank, receiver bank and gateway are simulated to demonstrate the concepts.

12. Common Mistakes to Avoid

- Updating **txn.state** directly and skipping the event row.
- Sharing one database session across threads.
- Forgetting to update balances together with the state change.
- Letting the demo depend on timing luck. Use the simulator switch so each failure is repeatable.
- Waiting for others. Publish early and fix later.